Engineering Predictable AI — Lecture Script (v6.2)

Companion to: `Slide_Deck_v6.2.pptx` (25 slides · light theme) **Class length:** 2h 30m **Audience:** Full-Stack devs · Data Analytics · GenAI — mixed cohort, assume no prior LLM knowledge **Version:** v6.2 · Jul 2026 · foundations expanded · reframe moved to earned pivot · 3 hands-on labs · interactive-first

---

How to use this script

Every slide gives you:

- **On screen** — a one-line reminder of what students see.
- **Time** — how long to stay. Never slip more than 2 min past target — cut and move on.
- **Say** — spoken script. Read once before class, then paraphrase.
- **Show / Do** — physical actions. Which tab to switch to, which file to open.
- **Interactive** — the one thing that makes this slide participatory (show of hands, predict-verify, or 30-sec micro-exercise). **Do not skip these.** They're 30 seconds each and they change retention 3x.
- **Analogy** — a non-tech comparison to reach the whole room, not just the developers.
- **Watch for** — the stumbles that happen every cohort.

Pre-class checklist (T-20 min)

On the projector laptop, tabs open left-to-right:

1. `Slide_Deck_v6.2.pptx` — presenter view, slide 1.
2. `claude.ai` — logged in, empty chat.
3. `console.anthropic.com/workbench` — logged in, empty prompt.
4. `chatgpt.com` — logged in, empty chat.
5. `aistudio.google.com` — logged in, empty prompt.
6. `hands-on-3-haystack.txt` — a shared doc for HANDS-ON #3 (see `assets/`).
7. A timer visible at top of screen.

Print or place at every seat:

- `Student_Cheat_Sheet_v6.2.pdf` (double-sided).
- A blank index card + pen (for the predict-and-bet moments).

30 seconds before start:

- Refresh playground tabs.
- Open the shared doc for HANDS-ON #3 in a browser tab.
- Take one deep breath. The first 5 minutes decide the whole class.

SEGMENT 0 — Foundations · what are we working with? (0:00 – 0:20)

Slide 1 — Engineering Predictable AI

On screen: Title. "Engineering Predictable AI. From 'type and hope' to a real workflow you can ship with."

Time: 1 minute.

Say:

> "Welcome. Two and a half hours. By the end of today, three things will be true. One, you'll know what an LLM actually is — not the marketing version, the mechanical version. Two, you'll have shipped a working prompt in the playground of your choice that returns strict JSON every single time. Three, you'll have a mental model that works with every AI tool you'll ever pick up — because tools change, but the primitives underneath don't."

Show / Do: Advance.

Interactive: *"Quick survey. Hands up if you've used ChatGPT this week. … Claude? … Gemini? … Now — hands up if you've been surprised by an answer it gave you in the last 7 days."* Every hand goes up on the last one. *"Good. That's why we're here."*

Watch for: Anyone with a closed laptop. Say: *"Open it. You'll need it in five minutes."*

Slide 2 — What is an LLM, really?

On screen: Token stream "The cat sat on the ___ [mat?]" with two cards: WHAT IT IS · WHAT IT ISN'T.

Time: 5 minutes. **Do not rush this.** Everything after depends on students believing this slide.

Say:

> "Before we do anything, we need to fix a misconception. Most people think an LLM is one of four things: a database, a search engine, an oracle, or a mind. It is none of those. Let me show you what it actually is."
>
> "An LLM is a **pattern-matching machine trained on internet text**. That's it. Given a bunch of tokens on the input, it predicts what token is most likely to come next. One token at a time. Do that 500 times in a row and you get a paragraph."
>
> "Look at the top of the slide. 'The cat sat on the ___.' Every human here just filled in the same word — mat. That's not intelligence, that's *pattern completion*. You've read that sentence a thousand times. So has the model. It knows what's likely next."
>
> "Now the right card. What the model **isn't**. Not a database — it doesn't have a lookup table of facts, it has patterns. Not a search engine — it can't go fetch things unless you plug in a tool. Not an oracle — when it doesn't know, it will guess plausibly rather than say 'I don't know.' That last one is the source of most hallucinations."
>
> "Punchline at the bottom of the slide. If you don't tell it, it doesn't know. Every fact you need in the answer has to be in the context. Otherwise the model will fill it in with something plausible-sounding — and sometimes wildly wrong."

Show / Do: Point at the "mat?" token. Say *"the model doesn't know 'mat' — it computes a probability distribution over every word in its vocabulary. Mat is highest. Then rug, floor, chair, mattress. It samples from that distribution. That's why the same prompt can give different answers."*

Interactive — Predict-verify (2 min): Type in the ChatGPT tab: *"What was the score of the Warriors vs Lakers game on July 3rd, 2026?"* Predict together: *"Show of hands — who thinks it will give you a real score? Who thinks it will make one up? Who thinks it will say it doesn't know?"* Run it. Almost always: it makes up a plausible-looking score. Land the point: *"That's not lying — that's the model doing what it does. Fill in likely-looking tokens."*

Analogy: *"An LLM is a writer with amnesia and no library card. It has read the internet, but it can't fact-check itself, and it forgets you between chats."*

Watch for: Somebody says *"but ChatGPT looked something up for me yesterday."* Answer: *"That was ChatGPT the product. It has a search tool plugged in. The LLM inside doesn't — the tool does. We'll cover that in the second hour."*

Slide 3 — What is a prompt, really?

On screen: 3D role stack — SYSTEM (teal) · USER (blue) · ASSISTANT (amber). Chat-box preview showing "you only see this half."

Time: 3 minutes.

Say:

> "You send a prompt. But the model doesn't see just what you typed. It sees a three-part envelope. **SYSTEM** — the rules, the persona, the format. **USER** — what you just asked. **ASSISTANT** — everything the model has already said in this conversation. All three go over the wire on every single API call."
>
> "The chat box shows you the USER part. It hides SYSTEM (which the vendor sets before you ever type anything) and it hides most of ASSISTANT (which it renders as scrolling text above your input box, but it's still there in the payload)."
>
> "The whole point of this class is to control all three roles. Not just the one the chat box gives you."

Show / Do: Open Claude Workbench in the projector. Point at the three boxes: *System*, *User*, *Assistant*. *"When you use a playground, you see the shape directly. That's why we're going to use playgrounds today, not chat.claude.com."*

Interactive — 30-sec micro-exercise: *"Everyone open ChatGPT. Send exactly this: 'You are a pirate. What's 2+2?' Watch the response. Now send just 'What's 2+2?' Compare. What changed?"* Someone will say *"it stopped talking like a pirate."* Land it: *"You just modified the SYSTEM-ish role via the USER role. In a playground, you'd set the persona in SYSTEM instead — cleaner, cheaper, and it persists automatically."*

Analogy: *"A prompt isn't a message. It's an envelope. Your text is one page in it. There's a cover letter from the vendor (SYSTEM) and a transcript of everything said before (ASSISTANT). The postman delivers the whole envelope."*

Watch for: Data-analytics students look confused when we talk about "roles." Say: *"Think of it like a CSV with three columns: role, content, order. Every API call is a small table like that."*

Slide 4 — The strict anatomy of an API call

On screen: JSON payload with model + system + messages array. Real-looking, not simplified.

Time: 3 minutes.

Say:

> "Here's what your innocent chat exchange actually is on the wire. Look at the code on the right. It's a POST to an HTTPS endpoint with a JSON body. The body has a `model` field, sometimes a top-level `system` field, and always a `messages` array. Each message has a `role` and a `content`. That's the entire API. Two fields per message. That's what you're paying for."
>
> "Anthropic, OpenAI, Google — they all use minor variations of this shape. Names differ. Anthropic uses `system` at the top level. OpenAI uses a `developer` message alongside `system`. Google calls it `system_instruction`. Same idea, three names. Once you can read one, you can read all three."

Show / Do: If any dev in the room has curl or Postman, do it live: run one real API call to Claude, show the response arrive. *"Every AI tool you'll ever use is a wrapper around this."*

Interactive: *"Look at the messages array. Point at every message where the model is speaking. … Now every message where the human is speaking. … Now — who set the `system` field?"* Answer: someone building the app did, before you ever loaded it.

Analogy: *"It's like reading the shipping label on a package. Once you know where 'from,' 'to,' and 'contents' are, you can decode any label from any carrier."*

Watch for: Non-devs glaze over at the JSON. Say: *"You don't have to write this. You have to be able to look at it and find the three roles. That's it."*

Slide 5 — What you CAN and CANNOT control

On screen: Three columns — YOU CONTROL (green) · YOU INFLUENCE (amber) · YOU DON'T CONTROL (pink).

Time: 5 minutes. **Critical reality-check slide.** Land it slowly.

Say:

> "Before we spend an hour on prompt engineering, know what game we're playing. Left column: things you fully control. System prompt, user message, model choice, temperature, max_tokens, tools, structured output schema, few-shot examples. Every one of these is a lever you turn on purpose."
>
> "Right column: things you don't control. Exact wording of the output — you can guide but not dictate. Internal reasoning steps — you can ask the model to show its work but you can't watch the actual computation. Whether the model 'really' used your context or fell back to training data. Latency and cost precision. And — importantly — the training cutoff date. The model doesn't know today. Ever."
>
> "Middle column: things you influence but can't guarantee. Format compliance — you can ask for JSON, but without structured output enforcement you'll get invalid JSON 3-10% of the time. Refusal behavior. Tone. How much reasoning it shows."
>
> "The takeaway at the bottom is the debugging rule of your career. When a prompt fails, first question: was it a control knob I forgot to turn, or an influence lever I can't force? The answer changes what you do next. If it was a control knob — go fix it. If it was influence — you might need to change your architecture entirely, not just the prompt."

Show / Do: Read each column top to bottom, slowly, tapping each item. Let it sink in.

Interactive — Show of hands (30 sec): *"Who's ever asked ChatGPT for JSON and gotten back JSON wrapped in prose or markdown fences? Show of hands."* Every dev raises. *"That's because you were on the INFLUENCE lever, not the CONTROL knob. In the next hour I'll show you the control knob."*

Analogy: *"Think of driving a car. You control the steering wheel, gas, brake, gear. You don't control the road conditions or other drivers. You influence your arrival time. Prompt engineering is knowing which is which so you stop yelling at the road."*

Watch for: Someone asks *"why can't we control the exact wording?"* Answer: *"Because the model samples from a probability distribution. Even at temperature=0, ties break inconsistently between providers. It's the nature of the beast. Learn to write prompts that don't care about exact wording."*

SEGMENT 1 — Better prompting = prompt engineering (0:20 – 1:00)

Slide 6 — Same three roles · three different surfaces

On screen: Comparison table across Claude Workbench, OpenAI Playground, AI Studio.

Time: 3 minutes.

Say:

> "You'll see this three-role pattern in three different playgrounds today. Names change, layouts change, but the boxes are always the same. Claude Workbench: System on the left, User in the middle. OpenAI Playground: developer message, user message, assistant message — labeled slots. AI Studio: System instructions in a panel on the right, prompt at the bottom. Learn the pattern once, use it everywhere. When Anthropic ships a new interface next month, you'll find System and User in 5 seconds because you know what you're looking for."

Show / Do: Cycle through the three playground tabs briefly. Point at the equivalent boxes in each.

Interactive: *"Pick whichever playground you like. Sign in if you haven't. You've got 90 seconds. Go."* Wait. Check the room. *"OK — everyone in? Good. That's your tool for the next hour."*

Analogy: *"It's like driving a manual car in Japan, then in Germany, then in the US. Pedals in different positions, gears labeled differently. But it's still: engage clutch, shift gear, release clutch. Same primitive."*

Watch for: Someone can't get into a playground (auth). Have them pair with a neighbor who's in — no one gets left behind for setup.

Slide 7 — The 6-ingredient prompt

On screen: 6 ingredients — ROLE · CONTEXT · TASK · CONSTRAINTS · OUTPUT FORMAT · EXAMPLES.

Time: 4 minutes.

Say:

> "Six things turn a vague prompt into a strong one. Not magic words — a specification checklist. Audit any prompt against these six. If you can't fill five of six, rewrite it before you send."
>
> "**Role.** Who is the model? Not 'you are helpful' — that's meaningless. 'You are a senior security engineer.' Specific."
>
> "**Context.** Who's the audience, what's the situation? 'My reader is a busy CFO who hates jargon.' "
>
> "**Task.** One verb. Classify. Extract. Summarize. If your task has 'and' in it, break it into two prompts."
>
> "**Constraints.** What must the model not do? Word limits. Tone limits. 'Never invent stats.' 'No marketing language.' "
>
> "**Output format.** Show a schema. Not describe one — show. `{ \"label\": \"...\", \"confidence\": 0.0-1.0 }`."
>
> "**Examples.** Zero, one, or a few. The model copies the *shape* of your examples more than the words. One good example is often worth 300 words of description."

Show / Do: For each ingredient, point at it on the slide and give a one-line example.

Interactive — Predict-verify (1 min): *"I'm going to write two prompts. Predict which one you'll trust to run 100 times in production."* Prompt A: *"tell me about AI"* Prompt B: *"You are a senior tech writer. Explain LLMs to a busy CFO in exactly 3 bullets, ≤ 20 words each. Never invent stats." "A? B? … Yeah. That's the whole class in one prompt pair."*

Analogy: *"Think of ordering a coffee. 'Give me a coffee' — you might get anything. 'Medium oat milk latte, one shot, extra hot, no foam' — six ingredients, you always get what you want. Same principle."*

Watch for: Someone says *"this feels like a lot of typing."* Answer: *"It's less typing than debugging the wrong output five times."*

Slide 8 — HANDS-ON #1 · Vague vs ingredient-complete (10 min)

On screen: Two-column task card — write a vague prompt, then rewrite with the 6 ingredients, compare.

Time: 10 minutes strict. Give timer.

Say:

> "OK — first hands-on. Ten minutes. Pick a task you actually care about. Write a deliberately vague prompt. Run it in your playground. Now rewrite it with the six ingredients. Run that. Compare the two outputs. Post your favorite comparison in chat. We'll review the best three."

Show / Do: Start the timer. Walk the room. Look over shoulders. Answer questions individually.

Interactive — the whole exercise IS the interactive part.

Paste — example task prompts if students freeze:

```
TASK IDEA (vague):
Write a summary of Q3 sales.

TASK IDEA (ingredient-complete):
You are a data analyst. Summarize the attached Q3 sales report for a busy
VP of Sales who has 60 seconds.
Return exactly 3 bullets, each ≤ 15 words.
Never invent numbers not in the source.
Format:
· [Trend]: [1-sentence explanation]
· [Winner]: [1-sentence explanation]
· [Concern]: [1-sentence explanation]
```

Watch for: Students who are typing verbose prompts and getting worse results. Coach them: *"Length isn't the goal. Specificity is."*

At time: *"OK — pens down. Two hands: whose ingredient-complete version was better? Everyone. Good. That's the whole lesson."*

Slide 9 — Prompt engineering · what & why

On screen: Definition + three "why" cards (repeatability · cost · trust).

Time: 3 minutes.

Say:

> "Now we have a name for what you just did. Prompt engineering is the discipline of writing prompts that produce **predictable** results across runs. The key word is predictable — same shape, same quality, run after run."
>
> "Three reasons it matters. **Repeatability** — same input, same output shape. **Cost** — fewer retries, cheaper API bills, faster iteration. **Trust** — you can put it in production without babysitting."
>
> "A prompt that returns valid JSON 60% of the time is a support ticket. A prompt that returns valid JSON 99.5% of the time is a shipping product. Prompt engineering is that difference."

Show / Do: Nothing new — just land the definition.

Interactive: *"Quick raise of hands — who's ever built a retry loop for AI output because it didn't parse? … A JSON repair library? … A regex to fix malformed responses?"* If any hand goes up: *"You've been paying the tax. Today we cut that tax to near zero."*

Analogy: *"It's the difference between a home-cooked meal and a franchise burger. The burger isn't better — it's the same every time. That's the discipline."*

Slide 10 — The right altitude for a prompt

On screen: Three cards — TOO VAGUE (pink) · RIGHT ALTITUDE (green) · TOO PRESCRIPTIVE (pink).

Time: 4 minutes.

Say:

> "There's a Goldilocks zone. **Too vague** — 'be helpful' — the model improvises and you get a coin toss. **Too prescriptive** — you write pseudo-code in English — the model can't deviate from your bad tree, and every case you didn't list breaks."
>
> "Read the middle card with me. 'Classify each message as SALES / SUPPORT / SPAM. Return JSON. Ask if empty.' Three sentences. That's the target. You defined the outcome and the shape. The model figures out the how."
>
> "The rule: define outcomes and shape. Let the model reason about the how. Your job is the goal, not the algorithm."

Show / Do: Read the too-prescriptive card in a monotone. The room laughs at *"IF billing AND refund AND $100 AND day = Monday…"*. Land it: *"That's what over-specification looks like. Every case you didn't list — Tuesday, US user, $99 — breaks."*

Interactive — 30-sec exercise: *"Look at your ingredient-complete prompt from HANDS-ON #1. Count your IF/THENs. Zero? Great. One? Fine. Two? Rewrite. Three or more? That's a red flag."*

Analogy: *"Ordering at a restaurant. TOO VAGUE: 'give me something good.' RIGHT ALTITUDE: 'I want a spicy vegetarian main, under $25.' TOO PRESCRIPTIVE: 'exactly 47g cauliflower, sautéed in olive oil for 4 minutes at 175°C, plated at 8 o'clock…' The chef can't cook. Same thing."*

Watch for: Someone asks *"how do I know when I'm too specific?"* Answer: *"If your prompt reads like a flowchart, you're too specific. Rewrite as a goal."*

Slide 11 — Force JSON · three providers, three syntaxes

On screen: Three code cards — Anthropic (tool_use / prefill), OpenAI (response_format), Google (response_mime_type).

Time: 5 minutes.

Say:

"This is the API-level fix for the 'JSON wrapped in markdown fences' problem you all raised earlier. Every provider has a native way to force structured JSON output — enforced at the API layer, not via prompting."

"Anthropic: use `tool_use` with an `input_schema`, or the prefill trick — you literally start the assistant turn with an opening brace character. Model has to continue from there."

"OpenAI: set `response_format: {type: 'json_schema', strict: true, schema: {...}}`. Guaranteed to match the schema. You can `JSON.parse` without a try-catch."

"Google: `response_mime_type: 'application/json'` plus `response_schema`. Same enforcement."

"There's also a universal fallback that works on any model — paste the schema in system, show one filled example, end the user turn with 'Return ONLY valid JSON. No prose, no fences.' Less reliable, but works everywhere."

Show / Do: In the OpenAI Playground, click through response_format → json_schema. Show the "schema" field. *"You paste your JSON schema here. That's it."*

Interactive: *"Everyone in a playground: find the structured-output setting. Where is it in yours?"* Wait. Point out the button in each of the three tools. This is orientation before HANDS-ON #2.

Analogy: *"Structured output is like asking the chef 'plate it on this exact platter' vs asking 'plate it however you want.' The platter fits the platter every time."*

Slide 12 — Grounding · remove the chance to guess

On screen: Grounding template — SYSTEM/USER pattern with source + quote + escape hatch.

Time: 4 minutes.

Say:

"If you want the model to answer from a source, don't just paste the source and hope. Force it. Three moves."

"One — paste the source in the prompt, inside clear delimiters like `<source>...</source>`. Two — tell the model to quote the sentence that supports each claim. Three — give it an escape hatch: 'If not in source, reply exactly: NOT_IN_SOURCE.' "

"That third move is the critical one. Without an escape hatch, the model has no legal way to say 'I don't know' — so it invents. Every RAG system in production uses some version of this pattern."

Show / Do: Point at the template on the slide. Read it aloud, slowly, once.

Interactive — Predict-verify: Type into Claude: *"What did Einstein say about time in his 1912 letter to Planck?"* (No source.) Predict: *"Will it make one up? Or say it doesn't know?"* Most times: it makes one up. Now retype WITH grounding: *"Answer ONLY from this source. If not in source, reply NOT_IN_SOURCE. [paste one Einstein biography paragraph you looked up] Question: What did Einstein say about time in his 1912 letter to Planck?"* Almost certainly returns NOT_IN_SOURCE. Land: *"You gave it an exit. It took it. Design for that."*

Analogy: *"Think of a court witness. Without ground rules, they'll speculate. With 'answer only from documents in evidence, or say I don't know' — they behave. Same for models."*

Watch for: Someone wants to know if this works for RAG apps. Answer: *"Yes. This IS how RAG works underneath. The retrieval fetches your source, the prompt template does the rest."*

Slide 13 — HANDS-ON #2 · Get strict JSON out (12 min)

On screen: Task — pick a provider, build a JSON-output prompt for a small bot, measure reliability.

Time: 12 minutes strict.

Say:

> "Twelve minutes. Pick a provider. Pick a small task — sentiment classifier, ticket router, movie-genre tagger, whatever. Write the prompt WITHOUT structured output first, run it 10 times, count how many are valid JSON. Then turn on structured output, run it 10 more times. Report your delta in chat."

Show / Do: Start timer. Walk. Coach individually.

Interactive — the exercise itself.

Paste — starter task if anyone freezes:

```
Classify this movie review as POSITIVE, NEGATIVE, or MIXED.
Also give a confidence between 0.0 and 1.0.
Return JSON: { "sentiment": "...", "confidence": ... }

Review: "The visuals were stunning but the plot dragged in the second act."
```

At time: Ask 3 students for their deltas. Almost always: prompt-only is 6-9/10, structured-output is 10/10. That's the win.

BREAK (1:00 – 1:10)

Encourage a real break. Water, bathroom, stretch. Do NOT run over.

SEGMENT 2 — The reframe · earned (1:10 – 1:20)

Slide 14 — You came for prompt engineering

On screen: The reframe card + Berglund quote + industry-consensus card. (This is the same slide that was slide 2 in v6.1, but it lands 10x harder now because they've EARNED it.)

Time: 5 minutes. **This is the emotional pivot of the whole class.** Slow down. Let it land.

Say:

> "You've just spent an hour on prompt engineering. You know how to write a six-ingredient prompt. You know how to force JSON. You know how to ground. You know what the model does and doesn't do."
>
> "So here's the update. The industry has moved past this."
>
> [pause 2 seconds — let it hang]
>
> "The course catalog still says Prompt Engineering. That's not wrong — everything you just learned still matters. But if you Google what Anthropic, Google, LangChain, and Confluent are publishing right now, none of them are calling it 'prompt engineering' anymore. They're calling it **context engineering**."
>
> "Read the quote on the left. Tim Berglund from Confluent, February 2026: *'Prompt engineering was always a little bit ridiculous. Context is king.'* That's not one person's take

— that's the whole industry converging in the last 12 months. Four independent players saying the same thing."

"Here's why they're right. A prompt is one string of text. The model doesn't see one string — it sees the whole envelope: system instructions, your message, retrieved documents, tool definitions, previous replies, tool results. **The prompt is one lever inside a system.** Everything we do for the rest of today is engineering the system."

Show / Do: Point at each of the four industry names as you read them. Land on the Berglund quote. Let the room breathe.

Interactive — Poll: *"Show of hands. How many of you signed up for this thinking it was going to be a class about writing better prompts? … How many now realize the field has moved?"* Every hand goes up on the second one. *"Good. You're getting the 2026 version, not the 2023 version."*

Analogy: *"You came in wanting to learn how to hit a golf ball. Turns out the whole sport is: pick the right club, read the wind, plan the shot, then hit. The swing is one piece. That's the reframe."*

Watch for: Someone anxious that they wasted an hour on prompts. Reassure: *"No — the prompt is the foundation of the pyramid we'll draw at the end. Everything you just learned is required. It's just not sufficient."*

Slide 15 — From prompts to context windows

On screen: Two-era card — 2022-2023 CHAT ERA (prompt) → 2024-2026 AGENT ERA (context window).

Time: 3 minutes.

Say:

"The shift, in one picture. In 2023, the unit of work was a prompt. You typed, the model replied, one turn. Skill = writing the right one prompt."

"In 2026, the unit is a context window. Models call tools, read retrieved docs, hold long conversations. Six things compete for space in every single call — the prompt is just one of them."

"Prompt engineering is still the foundation. Context engineering is built on top. We spent the last hour on the foundation. Now we spend an hour on what's built on top."

Show / Do: Point at 2023, then 2026. Slow.

Interactive: *"Think about the last time you used ChatGPT with a file upload. Or Claude with a project. How many things were in that context window besides your message?"* Let people answer: previous chat, uploaded file, custom instructions, retrieval results. *"Right. Six things. That's what we're about to name."*

Analogy: *"2023 was pen-pal correspondence — one letter at a time. 2026 is a shared Google Doc — multiple people, live cursors, comments, history, revisions all visible at once."*

SEGMENT 3 — Context engineering (1:20 – 2:05)

Slide 16 — The LLM is an OS · the context window is RAM

On screen: Two side-by-side stacks — LAPTOP (CPU/RAM/Disk) and AGENT (LLM/Context/Vector DB) — with dotted connectors.

Time: 5 minutes. This is the mental model for the whole second hour.

Say:

> "New mental model. Think of an LLM as an operating system. The **model** is the CPU — it does the reasoning. The **context window** is RAM — working memory, holds what the model can see NOW. External stores — files, databases, vector DBs — are your disk. Persistent, loaded into RAM only when you need them."
>
> "Punchline. When you open 50 Chrome tabs, your laptop crawls. Same failure in the model — when the context fills with junk, the reasoning degrades. Industry has a name for it: **context rot**. Everything we do for the rest of this hour is memory management. Which is oddly familiar territory if you've ever tuned an app."

Show / Do: Trace the dotted lines between CPU↔LLM, RAM↔Context, Disk↔Vector DB.

Interactive — Show of hands: *"Who's had 15 Chrome tabs open and watched their laptop crawl?"* Every hand. *"That's the same failure mode we're about to name in the model. You already have the intuition — I'm just going to give you the vocabulary."*

Analogy: *"The LLM is an office worker. The context window is their desk. External stores are the filing cabinet. If you dump the whole cabinet on the desk, they can't work. If you leave the desk empty, they don't have what they need. It's about desk management."*

Watch for: A dev pushes back: *"but the window is a million tokens, that's huge."* Answer: *"We'll get to that in three slides. Turns out models degrade WELL below the stated limit. Chroma tested 18 of them."*

Slide 17 — Anatomy of a context window · 6 competing components

On screen: Horizontal RAM-allocation bar — 6 segments split into STATIC ZONE (blue, LOCKED) and DYNAMIC ZONE (amber, GROWS).

Time: 5 minutes.

Say:

> "Six things fight for space every turn. Two you fully control up top: **System Prompt** and **Tool Definitions**. Four grow at the bottom: **User Message**, **Retrieved Resources**, **Assistant History**, and **Tool Calls & Responses**."
>
> "The blue half — STATIC ZONE — is where you put things that don't change turn-to-turn. Provider prefix-caches this half. If your system prompt is 2,000 tokens and stays the same across 100 API calls, prefix caching means you pay for those 2,000 tokens once, not 100 times. Anthropic offers this at 90% off cached tokens. That's real money."
>
> "The amber half — DYNAMIC ZONE — grows every turn. User adds a message. Model calls a tool, tool returns 5,000 tokens. Retrieval adds a document. By turn 10 of an agent run, the dynamic tail can bloat so much it suffocates your system prompt. That's the warning at the bottom."

Show / Do: Point at each segment left to right. Slow on the amber ones.

Interactive — Predict: *"If your system prompt says 'always reply in JSON,' and you run for 30 turns with lots of tool calls, guess what happens by turn 30 to the JSON compliance?"* Answer: degrades. *"Because the system prompt got buried under 40k tokens of tool noise. That's context rot in action."*

Analogy: *"Think of your fridge. Left side is the drawer with the yogurt you have every morning — stable, always there. Right side is leftovers and takeout — grows every week until you can't close the door. Same fridge, two very different management problems."*

Slide 18 — Four strategies · Write · Select · Compress · Isolate

On screen: 4 color-coded rows — each with a problem + a fix.

Time: 5 minutes.

Say:

> "LangChain's framework. Every single technique in the industry for managing context fits into one of these four buckets. Memorize the verbs — Write, Select, Compress, Isolate — and you have the whole field."
>
> "**Write.** Agents forget. When the window rolls over or gets summarized, information disappears. So you write things down outside the window. Scratchpads. A `claude.md` file that reloads every session. Memory tools that persist user preferences across days."
>
> "**Select.** Not everything is relevant right now. If your agent has 40 tools connected, you don't need all 40 in every call. Use RAG over the tool definitions. Load the right skill for the right step."
>
> "**Compress.** Context grows every turn. Summarize old conversation. Drop the raw 5,000-token webpage after you've extracted what you need."
>
> "**Isolate.** Contexts contaminate each other. If a single agent tries to research AND plan AND code AND debug in one conversation, the research phase pollutes the coding phase. Split into sub-agents. Reset context between phases."
>
> "That's the whole framework. When your agent misbehaves, ask: which of the four did I skip? The answer names itself."

Show / Do: Read each row. Point at problem, then fix.

Interactive: *"Look at the four verbs on screen. Everyone say them out loud together: Write, Select, Compress, Isolate."* Yes, this is corny. Yes, they'll remember it.

Analogy: *"Same four verbs you'd use with a paper desk. Write things down. Select what you need right now. Compress old notes into summaries. Isolate messy work to a separate corner."*

Slide 19 — Four ways context goes wrong

On screen: 2x2 grid — POISONING · DISTRACTION · CONFUSION · CLASH.

Time: 5 minutes.

Say:

> "When your agent misbehaves, it's almost never the model. It's one of these four named failure modes. Drew Bruyne named them in 2025 and the names stuck. Learn them — debugging gets 5x faster."
>
> "**Poisoning.** A hallucination or bad tool result gets into the context and every subsequent step references it. Fix: prune. Validate tool outputs before they enter context."
>
> "**Distraction.** The context gets so long the model over-fits to recent history, stops using its training. Fix: compress aggressively."
>
> "**Confusion.** Too many tools, model picks a nonsense one. LLaMA-3 with 46 tools failed a benchmark; the same model with only 19 relevant tools passed. Same task. Fix: select."
>
> "**Clash.** New info contradicts something already in the context. Model produces inconsistent output. Fix: authority order — system beats retrieved beats history. Say it explicitly."

Show / Do: Point at each quadrant. For each, gesture back to the strategy on slide 18 that fixes it.

Interactive: *"Think of the last time an AI tool did something weird for you. Was it Poisoning, Distraction, Confusion, or Clash?"* Give 20 seconds of thinking. Take one or two answers. Almost always fits one of the four.

Analogy: *"Same four failure modes a new employee has. Believes something wrong they were told → Poisoning. Fixates on today's fire, forgets training → Distraction. Overwhelmed by choice → Confusion. Two managers give conflicting orders → Clash. Fixes are the same for humans and agents."*

Slide 20 — Lost in the middle · the U-curve · the receipts

On screen: U-curve plot + The Receipts card (Chroma 2025 + Liu et al. + the 60-70% rule).

Time: 4 minutes.

Say:

> "This is not marketing. This is measured. Models weight tokens at the START and the END of the context heavier than the middle. The U-curve emerges from how attention works — every token attends to every other token, so as context grows, the ability to hold all pairwise relationships thins. Middle stuff gets dropped."
>
> "The receipts. Chroma, 2025 — 18 frontier models tested. Every single one degrades well before hitting the stated window limit. Liu et al. — 30+ percentage point accuracy drops when the same fact moves from position 1 to the middle. Same fact, same model, different position."
>
> "Practical rule: 60-70% of the window's real capacity. Bigger is not always better. A 200,000-token limit is a warning of where the model FAILS, not a target to hit. You're about to prove this yourself in the next hands-on."

Show / Do: Trace the U-curve with your finger. Land in the middle low point.

Interactive: *"Prediction time. If a fact is at position 1 vs position 500 vs position 999 out of 1000, which position wins by the most? Which loses?"* Answers: 1 wins, 500 loses hardest. Verify on next slide.

Analogy: *"Reading a 500-page book. You remember the opening chapter and the ending best. The middle — chapter 12 through 30 — is a blur. That's the U-curve for humans. Same math shape for models."*

Slide 21 — HANDS-ON #3 · Feel the U-curve (12 min)

On screen: 3-step exercise card — SETUP · RUN · COMPARE — with a BEFORE-YOU-RUN prediction bar.

Time: 12 minutes strict.

Say:

> "Twelve minutes. You're going to prove the U-curve on your own data. Setup step: open the shared doc 'hands-on-3-haystack.txt' — a 3,000-word article with a marked sentence at position 250, position 1500, and position 2900. Run step: paste the article into three separate chats. In chat A, ask about the START sentence. In chat B, the MIDDLE sentence. In chat C, the END sentence. Compare step: score each answer 0 to 2. Report your MIDDLE score in chat. We'll compute the class average live."
>
> "Before you run — everyone bet a coffee on which position will score lowest. Write it on your index card. If you're right, coffee's on the person to your left."

Show / Do: Start timer. Walk the room. Look over shoulders.

Interactive — the whole exercise IS the interactive part. The class scoring live is the payoff moment.

Paste — the exact haystack file structure students will see:

```
[haystack file structure — provided in assets/hands-on-3-haystack.txt]

Article begins here...
[Position 250 — START sentence marked: "The launch of the Voyager 3 mission was
approved by the ESA on March 15, 2029."]
... 700 words of article ...
[Position 1500 — MIDDLE sentence marked: "The Voyager 3 primary science target
is the Kuiper Belt object 486958 Arrokoth."]
... 700 words of article ...
[Position 2900 — END sentence marked: "First data return from Voyager 3 is
scheduled for October 2035."]
```

At time: Ask 5 students for their MIDDLE score. Compute the class average live. Compare to their START/END scores. **The U-curve will appear.** Land it: *"Not a study. Your data. You just measured context rot."*

Slide 22 — The 3-phase loop · with context reset barriers

On screen: Three phase cards — RESEARCH → PLAN → IMPLEMENT — with pink CONTEXT RESET bars between.

Time: 5 minutes.

Say:

> "Everything we just covered — write, select, compress, isolate — comes together as this one pattern. The most reliable pattern for any long agent task. Human Layer shipped 35,000 lines of Rust in one 7-hour session using exactly this."
>
> "Three phases. **Research.** Deep read. Map the terrain. No assumptions. Output: research.md. **Plan.** Fresh window, only research.md and the problem definition, produces plan.md. This is where YOU annotate. This is where fixing a logical error is a two-line note instead of a two-hour debugging session. **Implement.** Fresh window again, only plan.md, execute step by step. Typecheck as you go."
>
> "Notice the pink bars. Context reset barriers. You don't just keep chatting — you open a NEW window with only the compacted artifact from the previous phase. Zero contamination. The research phase's messy 60,000 tokens never pollute the implementation phase."

Show / Do: Point at each phase. Slow on the middle bar — that's where the human annotation happens; that's the leverage point.

Interactive: *"Ask: which of the four strategies is happening at each context-reset barrier?"* Answer: Compress + Isolate. *"Yes. The whole framework in action."*

Analogy: *"Writing a book. You outline (research). You get a critique letter from the editor (plan/annotation). You write the book from the outline, not from the pile of research notes (implement). Every author uses this loop. Agents do too."*

SEGMENT 4 — Ship it (2:05 – 2:30)

Slide 23 — 10 tips to pin to your wall

On screen: 10 numbered one-liners in 2 columns.

Time: 6 minutes.

Say:

> "The whole class in ten rules-of-thumb. Print this slide. Pin it. Reread every Monday."
>
> [Read each tip aloud. Add a one-sentence example for tips 4, 5, 6, 8. Don't rush.]
>
> "The meta-tip at the bottom is the one to actually memorize. Name the primitive — prompt, context, or workflow — before you touch it. It changes what you try next by 5x."

Show / Do: Read each tip slowly. Pause between tips.

Interactive — 30-sec commitment: *"Everyone: pick the ONE tip you personally are most likely to forget. Circle it on the cheat sheet at your seat. This is the one you tape to your monitor tonight."*

Analogy: *"These are the 'wet paint' signs of AI engineering. Boring, obvious, and if you ignore them you ruin the coat."*

Slide 24 — The hierarchy of AI engineering

On screen: 3-tier pyramid — Prompt Formulation (base) · Context Management (middle) · Architected Workflows (top).

Time: 4 minutes.

Say:

> "Zoom out. Three tiers. **Prompt formulation** — the base. Roles, JSON schemas, grounding, escape hatches. **Context management** — the middle. U-curve, 4 strategies, failure modes, compaction. **Architected workflows** — the top. Research/plan loops, sub-agents, MCP."
>
> "If the bottom tier fails, everything above it fails. A great autonomous session isn't magic. It's great prompt formulation, scaled by context management, wrapped in workflow architecture. This is why we started at the base today."
>
> "Prompt is the atom. Context is the molecule. Workflow is the organism. You're here to master the atom first."

Show / Do: Trace up the pyramid: TEAL → VIOLET → AMBER. Land on the base — *"start here."*

Interactive: *"Where are you tonight? Bottom, middle, or top?"* Take a few answers. Almost everyone is at the bottom. That's correct.

Analogy: *"Cooking. Base = knife skills and heat control. Middle = seasoning and timing. Top = menu planning and plating for guests. Skip the base, and no amount of plating saves the dish."*

Slide 25 — Cheat sheet · what's next · Q&A

On screen: Cheat sheet reminder + next courses + Q&A prompt.

Time: ~15 minutes for real Q&A.

Say:

"You have a cheat sheet at your seat. Pin it near your monitor. It has the whole class on one page — reframe, three roles, six ingredients, right altitude, four strategies, four failure modes, three-phase loop, and the ten tips."

"What's next in the bootcamp. **MCP day** — how tools plug into models. **Agentic AI week** — how to orchestrate multiple agents. **Building LLM Apps** — how to ship this stuff to production. Every one of them builds on today."

"Two and a half hours ago, you came for prompt engineering. You're leaving with context engineering — vocabulary, mental model, one workflow you can copy tomorrow. Questions?"

Show / Do: Point at the cheat sheet. Take questions. Circle back to anyone who was stuck earlier.

Interactive — real Q&A. Common questions to prep:

- *"Which tool should I start with?"* → Whichever one you already have an account with. All three (Claude, ChatGPT, Gemini) do the same three things.
- *"Does this apply to fine-tuning too?"* → Yes. Fine-tuning changes the model's weights; everything on the pyramid still applies to how you use it.
- *"How much does the API cost?"* → For learning, budget $5-20/month. Structured output + prefix caching keeps costs way lower than most people think.
- *"When will the industry rename this again?"* → Probably within 18 months. Don't chase the name — chase the primitives underneath it.

Close:

"One line to walk out with. The prompt is the atom. The context window is the molecule. The workflow is the organism. Start with atoms. Ship. Iterate. See you next week."